

Graph theory in OI

whx

February 23, 2018

网络流和费用流

■ 定义

网络流和费用流

- 定义
- 割的定义

网络流和费用流

- 定义
- 割的定义
- 最大流最小割定理

网络流和费用流

- 定义
- 割的定义
- 最大流最小割定理
- 上下界网络流的建模（后面会用到）

网络流和费用流

- 定义
- 割的定义
- 最大流最小割定理
- 上下界网络流的建模（后面会用到）
- dinic 与 unit graph

二分图匹配和霍尔定理

■ 霍尔定理

正则二分图

- 每个点的度均为 k 的二分图

正则二分图

- 每个点的度均为 k 的二分图
- 一定可以拆成 k 个匹配的并!

- 每个点度数都为 3 的二分图，分割成若干个简单环使得每个边恰好在两个简单环中。

- 每个点度数都为 3 的二分图，分割成若干个简单环使得每个边恰好在两个简单环中。
- $n \leq 10^5$

- 每个点度数都为 3 的二分图，分割成若干个简单环使得每个边恰好在两个简单环中。
- $n \leq 10^5$
- 先拆成三个匹配，然后两两拼一起求欧拉回路。

- $n * n$ 的棋盘上摆了 $n \leq 10^5$ 个车，让他们两两不攻击

POJ chessboard

- $n * n$ 的棋盘上摆了 $n \leq 10^5$ 个车，让他们两两不攻击
- 第 i 个车必须摆在一个矩形里

- $n * n$ 的棋盘上摆了 $n \leq 10^5$ 个车，让他们两两不攻击
- 第 i 个车必须摆在一个矩形里
- 求哪些车的摆放位置是确定的？

■ 行列独立

POJ chessboard

- 行列独立
- 二分图匹配

POJ chessboard

- 行列独立
- 二分图匹配
- 每个点匹配是否唯一?

- 行列独立
- 二分图匹配
- 每个点匹配是否唯一?
- 往右只保留匹配边，往左只保留未匹配边，它是不是在一个环中

- 行列独立
- 二分图匹配
- 每个点匹配是否唯一?
- 往右只保留匹配边，往左只保留未匹配边，它是不是在一个环中
- 缩强连通分量即可

- 边 i , 边权 $+1$ 有代价 a_i , -1 有代价 b_i

- 边 i , 边权 $+1$ 有代价 a_i , -1 有代价 b_i
- 修改边权使得指定的生成树变为最小生成树, 求最小代价

- 边 i , 边权 $+1$ 有代价 a_i , -1 有代价 b_i
- 修改边权使得指定的生成树变为最小生成树, 求最小代价
- $n \leq 300$

- 边 i , 边权 $+1$ 有代价 a_i , -1 有代价 b_i
- 修改边权使得指定的生成树变为最小生成树, 求最小代价
- $n \leq 300$
- 显然只会给生成树树边减, 给非树边加, 可以列出方程
$$x_i + y_j \geq \delta$$

- 边 i , 边权 $+1$ 有代价 a_i , -1 有代价 b_i
- 修改边权使得指定的生成树变为最小生成树, 求最小代价
- $n \leq 300$
- 显然只会给生成树树边减, 给非树边加, 可以列出方程
$$x_i + y_j \geq \delta$$
- 最小顶标和, 对偶一下。

- 有 n 个题目的题库，每个题的难度为 c_i 。从中出 m 个题目，第 i 题难度需要在 a_i 到 b_i 之间

CCPC E Problem Buyer

- 有 n 个题目的题库，每个题的难度为 c_i 。从中出 m 个题目，第 i 题难度需要在 a_i 到 b_i 之间
- 求最小的 k 使得对于任意一个 k 个题目的子集都能凑出一套符合要求的题。

CCPC E Problem Buyer

- 有 n 个题目的题库，每个题的难度为 c_i 。从中出 m 个题目，第 i 题难度需要在 a_i 到 b_i 之间
- 求最小的 k 使得对于任意一个 k 个题目的子集都能凑出一套符合要求的题。
- $n, m \leq 10^5$

- 合法条件? 每个子区间都满足霍尔定理

CCPC E Problem Buyer

- 合法条件? 每个子区间都满足霍尔定理
- 扫描线 + 线段树维护

Delight for a cat

- 有一只喵，共 n 天，它每 k 天吃吃的天数要在 $[l, r]$ 之间，每天睡觉有个收益 s_i ，吃猫粮有个收益 e_i ，求最大收益。

Delight for a cat

- 有一只猫，共 n 天，它每 k 天吃吃的天数要在 $[l, r]$ 之间，每天睡觉有个收益 s_i ，吃猫粮有个收益 e_i ，求最大收益。
- $k, n \leq 1000$

Delight for a cat

- 有一只猫，共 n 天，它每 k 天吃吃的天数要在 $[l, r]$ 之间，每天睡觉有个收益 s_i ，吃猫粮有个收益 e_i ，求最大收益。
- $k, n \leq 1000$
- 输出方案

线性规划转网络流

- 解法 1: 每个限制的天数都是连续的, 可以对偶差分

线性规划转网络流

- 解法 1: 每个限制的天数都是连续的, 可以对偶差分
- 解法 2: 每天在限制上也是连续的, 可以直接差分不对偶

负权环建图

- 考虑每天对于限制是一个连续区间

负权环建图

- 考虑每天对于限制是一个连续区间
- 我们从 $i+k+1$ 连到 i , 代价为 $e_i - s_i$, 然后 i 往 $i+1$ 连上下界为 $[l, r]$ 的边

负权环建图

- 考虑每天对于限制是一个连续区间
- 我们从 $i+k+1$ 连到 i , 代价为 $e_i - s_i$, 然后 i 往 $i+1$ 连上下界为 $[l, r]$ 的边
- 这是个非常容易拓展出题的 trick, 解决了直接从 S 和 T 连不匹配的问题

负权环建图

- 考虑每天对于限制是一个连续区间
- 我们从 $i+k+1$ 连到 i , 代价为 $e_i - s_i$, 然后 i 往 $i+1$ 连上下界为 $[l, r]$ 的边
- 这是个非常容易拓展出题的 trick, 解决了直接从 S 和 T 连不匹配的问题
- 比如 DAG 上选若干路径, 最大化他们的并的权值和减去他们的权值和

负权建图技巧

■ 预先流满

集训队集训 day1 t2

- $nm \leq 1000$ 的网格图，里面有一些水管。

集训队集训 day1 t2

- $nm \leq 1000$ 的网格图，里面有一些水管。
- 水管可以有很多头 (2 到 4 个)，其中直线的水管不可以转，别的水管都可以以 1 的代价旋转九十度，可以顺时针或者逆时针转。

集训队集训 day1 t2

- $nm \leq 1000$ 的网格图，里面有一些水管。
- 水管可以有很多头 (2 到 4 个)，其中直线的水管不可以转，别的水管都可以以 1 的代价旋转九十度，可以顺时针或者逆时针转。
- 请问最少多少词旋转使得没有漏水？

- 比较明显的匹配模型

集训队集训 day1 t2

- 比较明显的匹配模型
- 首先黑白染色分出 S 集和 T 集

集训队集训 day1 t2

- 比较明显的匹配模型
- 首先黑白染色分出 S 集和 T 集
- 然后分类讨论

集训队集训 day1 t2

- 如果是两个头的 L 形，那么限制匹配的一定是一个行点和一个列点

集训队集训 day1 t2

- 如果是两个头的 L 形，那么限制匹配的一定是一个行点和一个列点
- 然后行点和列点变化代价分别为 1

集训队集训 day1 t2

- 如果是两个头的 L 形，那么限制匹配的一定是一个行点和一个列点
- 然后行点和列点变化代价分别为 1
- 如果是三个头就更简单了，给没选的那个带一个负数权值，表示没选有个代价

集训队集训 day1 t2

- 如果是两个头的 L 形，那么限制匹配的一定是一个行点和一个列点
- 然后行点和列点变化代价分别为 1
- 如果是三个头就更简单了，给没选的那个带一个负数权值，表示没选有个代价
- 加一个常数把它变成正的。

- 给你一个二分图，每侧 n 个点

- 给你一个二分图，每侧 n 个点
- 多少个点的子集存在完美匹配

- 给你一个二分图，每侧 n 个点
- 多少个点的子集存在完美匹配
- $n \leq 20$

- 回忆一下线代方法的证明，反对称矩阵的性质

- 回忆一下线代方法的证明，反对称矩阵的性质
- 不难猜想存在完美匹配的充要条件是左边和右边的子集分别满足霍尔定理

- 回忆一下线代方法的证明，反对称矩阵的性质
- 不难猜想存在完美匹配的充要条件是左边和右边的子集分别满足霍尔定理
- check 霍尔定理? mask dp

TCO SemiFinal Colorful Tree

- 一个 DAG，满足 $u \rightarrow v$ 的边中 $u < v$ ，并且满足边只有包含没有相交（顶点处相交不算）

TCO SemiFinal Colorful Tree

- 一个 DAG，满足 $u \rightarrow v$ 的边中 $u < v$ ，并且满足边只有包含没有相交（顶点处相交不算）
- 点有颜色，求一条 1 到 n 的路径

TCO SemiFinal Colorful Tree

- 一个 DAG，满足 $u \rightarrow v$ 的边中 $u < v$ ，并且满足边只有包含没有相交（顶点处相交不算）
- 点有颜色，求一条 1 到 n 的路径
- 每个颜色的点要么全部经过要么全部不经过，边有边权，求最短的这条路径

TCO SemiFinal Colorful Tree

- 一个 DAG，满足 $u \rightarrow v$ 的边中 $u < v$ ，并且满足边只有包含没有相交（顶点处相交不算）
- 点有颜色，求一条 1 到 n 的路径
- 每个颜色的点要么全部经过要么全部不经过，边有边权，求最短的这条路径
- $n \leq 50$

TCO SemiFinal Colorful Tree

- 边只有包含没有相交其实构成了一棵树

TCO SemiFinal Colorful Tree

- 边只有包含没有相交其实构成了一棵树
- 这个图的最短路也就是对偶图（树）的最小割

TCO SemiFinal Colorful Tree

- 边只有包含没有相交其实构成了一棵树
- 这个图的最短路也就是对偶图（树）的最小割
- 可以连 inf 边来限制两个点同时在 S/T 集

TCO SemiFinal Colorful Tree

- 边只有包含没有相交其实构成了一棵树
- 这个图的最短路也就是对偶图（树）的最小割
- 可以连 inf 边来限制两个点同时在 S/T 集
- 连 inf 边会影响形态，每个点要往父亲连 inf 边确保每个路径只割一刀

- 有一些点 (x, y) ，请你对点黑白染色，使得每行每列都满足 $|blackpoint - whitepoint| \leq 1$

- 有一些点 (x, y) ，请你对点黑白染色，使得每行每列都满足 $|blackpoint - whitepoint| \leq 1$
- $n \leq 2 * 10^5$

- 有一些点 (x, y) ，请你对点黑白染色，使得每行每列都满足 $|blackpoint - whitepoint| \leq 1$
- $n \leq 2 * 10^5$
- 如果白色点个数等于黑色点个数怎么做呢？

- 有一些点 (x, y) ，请你对点黑白染色，使得每行每列都满足 $|blackpoint - whitepoint| \leq 1$
- $n \leq 2 * 10^5$
- 如果白色点个数等于黑色点个数怎么做呢？
- 行列连边连出一个二分图，求欧拉回路黑白染色

- 有一些点 (x, y) , 请你对点黑白染色, 使得每行每列都满足 $|blackpoint - whitepoint| \leq 1$
- $n \leq 2 * 10^5$
- 如果白色点个数等于黑色点个数怎么做呢?
- 行列连边连出一个二分图, 求欧拉回路黑白染色
- 那么差小于等于 1 我们可以每个奇数行列加一个边使得它存在欧拉回路

- 有一些点 (x, y) , 请你对点黑白染色, 使得每行每列都满足 $|blackpoint - whitepoint| \leq 1$
- $n \leq 2 * 10^5$
- 如果白色点个数等于黑色点个数怎么做呢?
- 行列连边连出一个二分图, 求欧拉回路黑白染色
- 那么差小于等于 1 我们可以每个奇数行列加一个边使得它存在欧拉回路
- 二分图两边的奇数点的个数不一定一样多, 所以应该两边建两个虚点, 然后连到虚点

- 有一些区间 $[li, ri]$, 请你对这些区间黑白染色, 使得每个点覆盖它的 $|blacksegment - whitesegment| \leq 1$

- 有一些区间 $[li, ri]$, 请你对这些区间黑白染色, 使得每个点覆盖它的 $|blacksegment - whitesegment| \leq 1$
- $n \leq 10^5$

- 有一些区间 $[li, ri]$, 请你对这些区间黑白染色, 使得每个点覆盖它的 $|blacksegment - whitesegment| \leq 1$
- $n \leq 10^5$
- 考虑如果是等于, 那么就是每个点的被覆盖次数 s_i 等于 0

- 有一些区间 $[li, ri]$, 请你对这些区间黑白染色, 使得每个点覆盖它的 $|blacksegment - whitesegment| \leq 1$
- $n \leq 10^5$
- 考虑如果是等于, 那么就是每个点的被覆盖次数 s_i 等于 0
- 那么我们差分一下这个 s_i , 那么每个区间的贡献就是 a_l++ , $a_{r+1}--$, 或者 a_l-- , $a_{r+1}++$

- 有一些区间 $[l_i, r_i]$, 请你对这些区间黑白染色, 使得每个点覆盖它的 $|blacksegment - whitesegment| \leq 1$
- $n \leq 10^5$
- 考虑如果是等于, 那么就是每个点的被覆盖次数 s_i 等于 0
- 那么我们差分一下这个 s_i , 那么每个区间的贡献就是 a_l++ , $a_{r+1}--$, 或者 a_l-- , $a_{r+1}++$
- 连一条 l 和 $r+1$ 之间的边, 求欧拉回路黑白染色

- 有一些区间 $[l_i, r_i]$, 请你对这些区间黑白染色, 使得每个点覆盖它的 $|blacksegment - whitesegment| \leq 1$
- $n \leq 10^5$
- 考虑如果是等于, 那么就是每个点的被覆盖次数 s_i 等于 0
- 那么我们差分一下这个 s_i , 那么每个区间的贡献就是 a_l++ , $a_{r+1}--$, 或者 a_l-- , $a_{r+1}++$
- 连一条 l 和 $r+1$ 之间的边, 求欧拉回路黑白染色
- 大于等于 1 怎么做呢? 这里可以把奇数点两两配对了

- 二分图上不能走过重复的点，每人走一步，谁会获胜？

- 二分图上不能走过重复的点，每人走一步，谁会获胜？
- 经典问题

- 二分图上不能走过重复的点，每人走一步，谁会获胜？
- 经典问题
- 先手必胜当且仅当起点一定在最大匹配上

- 二分图上不能走过重复的点，每人走一步，谁会获胜？
- 经典问题
- 先手必胜当且仅当起点一定在最大匹配上
- 点是否一定在最大匹配上？

- 二分图上不能走过重复的点，每人走一步，谁会获胜？
- 经典问题
- 先手必胜当且仅当起点一定在最大匹配上
- 点是否一定在最大匹配上？
- bfs 有无增广轨

UNR 奇怪的线段树

- 按照任意顺序分治形成的线段树

UNR 奇怪的线段树

- 按照任意顺序分治形成的线段树
- 定位一个区间会经过若干区间把他们全部染黑

UNR 奇怪的线段树

- 现在定位了一些区间，给你染黑的情况，请你求出至少要定位多少个区间才能变成这样。

UNR 奇怪的线段树

- 现在定位了一些区间，给你染黑的情况，请你求出至少要定位多少个区间才能变成这样。
- $n \leq 4000$

UNR 奇怪的线段树

- 显然把所有染黑部分的叶子覆盖了即可

UNR 奇怪的线段树

- 显然把所有染黑部分的叶子覆盖了即可
- 做一些观察，一个合法的染黑区间一定是不能同时选某个点的左儿子和右儿子并且连续的路径

UNR 奇怪的线段树

- 显然把所有染黑部分的叶子覆盖了即可
- 做一些观察，一个合法的染黑区间一定是不能同时选某个点的左儿子和右儿子并且连续的路径
- 每个左孩子的后继可以连到和他连续并且深度大于它的所有节点（还会是一个左孩子）

UNR 奇怪的线段树

- 显然把所有染黑部分的叶子覆盖了即可
- 做一些观察，一个合法的染黑区间一定是不能同时选某个点的左儿子和右儿子并且连续的路径
- 每个左孩子的后继可以连到和他连续并且深度大于它的所有节点（还会是一个左孩子）
- 每个右孩子的后继可以连到任意一个和他连续的节点

UNR 奇怪的线段树

- 显然把所有染黑部分的叶子覆盖了即可
- 做一些观察，一个合法的染黑区间一定是不能同时选某个点的左儿子和右儿子并且连续的路径
- 每个左孩子的后继可以连到和他连续并且深度大于它的所有节点（还会是一个左孩子）
- 每个右孩子的后继可以连到任意一个和他连续的节点
- 所以路径一定是一段右儿子一段左儿子

UNR 奇怪的线段树

- 建图的时候把左孩子和自己的左孩子串起来，并且把每个点串到左端点上即可

UNR 奇怪的线段树

- 建图的时候把左孩子和自己的左孩子串起来，并且把每个点串到左端点上即可
- 然后变成了用最少的路径覆盖一个 DAG 的问题

UNR 奇怪的线段树

- 建图的时候把左孩子和自己的左孩子串起来，并且把每个点串到左端点上即可
- 然后变成了用最少的路径覆盖一个 DAG 的问题
- 上下界网络流即可

DAG 的传递闭包近似

- 一个 n 个点, m 条边的 DAG

DAG 的传递闭包近似

- 一个 n 个点, m 条边的 DAG
- 求每个点的传递闭包的大小, 两倍近似

DAG 的传递闭包近似

- minhash trick

DAG 的传递闭包近似

- minhash trick
- 考虑 n 个 0 到 1 之间的独立的随机变量最小值的期望（推荐播客，从零道一）

DAG 的传递闭包近似

- minhash trick
- 考虑 n 个 0 到 1 之间的独立的随机变量最小值的期望（推荐播客，从零道一）
- 比 x 大的概率是 $(1 - x)^n$

DAG 的传递闭包近似

- minhash trick
- 考虑 n 个 0 到 1 之间的独立的随机变量最小值的期望（推荐播客，从零道一）
- 比 x 大的概率是 $(1 - x)^n$
- $\int_0^1 (1 - x)^n = \int_0^1 x^n = \frac{1}{n+1}$

DAG 的传递闭包近似

- minhash trick
- 考虑 n 个 0 到 1 之间的独立的随机变量最小值的期望（推荐播客，从零道一）
- 比 x 大的概率是 $(1 - x)^n$
- $\int_0^1 (1 - x)^n = \int_0^1 x^n = \frac{1}{n+1}$
- 用期望估计答案！

生成树形图

- 求 root 为 n 的生成树形图的所有方案的边权和之和

生成树形图

- 求 root 为 n 的生成树形图的所有方案的边权和之和
- 这种计数问题（包括积和式转行列式求二分图匹配奇偶性的题），我们强制必须选缩起来不太好做（虽然这题等于两个列向量相加，然后就可以做了 QAQ）

生成树形图

- 求 root 为 n 的生成树形图的所有方案的边权和之和
- 这种计数问题（包括积和式转行列式求二分图匹配奇偶性的题），我们强制必须选缩起来不太好做（虽然这题等于两个列向量相加，然后就可以做了 QAQ）
- 但是这种问题更简单的考虑办法是，考虑用生成树个数减去删去这个边之后的生成树个数

生成树形图

- 求 root 为 n 的生成树形图的所有方案的边权和之和
- 这种计数问题（包括积和式转行列式求二分图匹配奇偶性的题），我们强制必须选缩起来不太好做（虽然这题等于两个列向量相加，然后就可以做了 QAQ）
- 但是这种问题更简单的考虑办法是，考虑用生成树个数减去删去这个边之后的生成树个数
- 把这个边删去（两个地方-1），然后用原来的行列式减去删去之后的行列式，就可以算了

生成树形图

- 求 root 为 n 的生成树形图的所有方案的边权和之和
- 这种计数问题（包括积和式转行列式求二分图匹配奇偶性的题），我们强制必须选缩起来不太好做（虽然这题等于两个列向量相加，然后就可以做了 QAQ）
- 但是这种问题更简单的考虑办法是，考虑用生成树个数减去删去这个边之后的生成树个数
- 把这个边删去（两个地方-1），然后用原来的行列式减去删去之后的行列式，就可以算了
- 注意这题里这两个地方在同一行，然后我们把行列式关于这行展开，就可以变成求个逆，算伴随矩阵的问题了

生成树形图

- 求 root 为 n 的生成树形图的所有方案的边权和之和
- 这种计数问题（包括积和式转行列式求二分图匹配奇偶性的题），我们强制必须选缩起来不太好做（虽然这题等于两个列向量相加，然后就可以做了 QAQ）
- 但是这种问题更简单的考虑办法是，考虑用生成树个数减去删去这个边之后的生成树个数
- 把这个边删去（两个地方-1），然后用原来的行列式减去删去之后的行列式，就可以算了
- 注意这题里这两个地方在同一行，然后我们把行列式关于这行展开，就可以变成求个逆，算伴随矩阵的问题了
- 需要注意伴随矩阵其实是代数余子式构成的矩阵的转置。。需要反一下 i 和 j

- 给两个串，求第二个串的每个子串和第一个串的 LCS

- 给两个串，求第二个串的每个子串和第一个串的 LCS
- $O(n^2)$

- ALCS 大概就是转成网格图的最长路，把网格图称为 GDAG

- ALCS 大概就是转成网格图的最长路，把网格图称为 GDAG
- $f[l][i][j]$ 表示 $A[1..l]$ 和 $B[i..j]$ 的 LCS，对应的是起点在 $(i, 0)$ 终点在 (j, l) 的最长路

- ALCS 大概就是转成网格图的最长路，把网格图称为 GDAG
- $f[l][i][j]$ 表示 $A[1..l]$ 和 $B[i..j]$ 的 LCS，对应的是起点在 $(i, 0)$ 终点在 (j, l) 的最长路
- 我们考虑 $j++$ 的时候，一个 j 对应的所有 i 哪些会 $+1$ ，考虑终点为 (j, l) 的最长路树，因为路径不相交，所以显然我们可以取到一个 leftmost 的路径

- ALCS 大概就是转成网格图的最长路，把网格图称为 GDAG
- $f[l][i][j]$ 表示 $A[1..l]$ 和 $B[i..j]$ 的 LCS，对应的是起点在 $(i, 0)$ 终点在 (j, l) 的最长路
- 我们考虑 $j++$ 的时候，一个 j 对应的所有 i 哪些会 $+1$ ，考虑终点为 (j, l) 的最长路树，因为路径不相交，所以显然我们可以取到一个 leftmost 的路径
- 那么如果一个 i 可以从 $(j-1, l)$ 转移过来，那么说明 $f[i][j-1][l] = f[i][j][l]$ ，所以 $j++$ 的时候， $f++$ 对应的 i 肯定是一个后缀区间

- 同样 $l++$ 的时候，取 rightmost 的路径可以尽量从 $(j, l-1)$ 转移过来，所以 $l++$ 的时候对应的是一个前缀

- 同样 $l++$ 的时候，取 rightmost 的路径可以尽量从 $(j, l-1)$ 转移过来，所以 $l++$ 的时候对应的是一个前缀
- 同样 $l++$ 的时候，取 rightmost 的路径可以尽量从 $(j, l-1)$ 转移过来，所以 $l++$ 的时候对应的是一个前缀

- 同样 $l++$ 的时候, 取 rightmost 的路径可以尽量从 $(j, l-1)$ 转移过来, 所以 $l++$ 的时候对应的是一个前缀
- 同样 $l++$ 的时候, 取 rightmost 的路径可以尽量从 $(j, l-1)$ 转移过来, 所以 $l++$ 的时候对应的是一个前缀
- 我们把 $(w, h-1) \rightarrow (w, h)$ 的 $+1$ 前缀称为 $\text{pre}[w][h]$, 同样的 $(w-1, h) \rightarrow (w, h)$ 的 $+1$ 后缀称为 $\text{suf}[w][h]$

- 同样 $l++$ 的时候, 取 *rightmost* 的路径可以尽量从 $(j, l-1)$ 转移过来, 所以 $l++$ 的时候对应的是一个前缀
- 同样 $l++$ 的时候, 取 *rightmost* 的路径可以尽量从 $(j, l-1)$ 转移过来, 所以 $l++$ 的时候对应的是一个前缀
- 我们把 $(w, h-1) \rightarrow (w, h)$ 的 $+1$ 前缀称为 $pre[w][h]$, 同样的 $(w-1, h) \rightarrow (w, h)$ 的 $+1$ 后缀称为 $suf[w][h]$
- 然后如果 $a[w] == b[h]$, 那么从 $(w-1, h-1)$ 到 (w, h) 的时候所有 i 都 $+1$ 了

- 同样 $l++$ 的时候, 取 rightmost 的路径可以尽量从 $(j, l-1)$ 转移过来, 所以 $l++$ 的时候对应的是一个前缀
- 同样 $l++$ 的时候, 取 rightmost 的路径可以尽量从 $(j, l-1)$ 转移过来, 所以 $l++$ 的时候对应的是一个前缀
- 我们把 $(w, h-1) \rightarrow (w, h)$ 的 $+1$ 前缀称为 $\text{pre}[w][h]$, 同样的 $(w-1, h) \rightarrow (w, h)$ 的 $+1$ 后缀称为 $\text{suf}[w][h]$
- 然后如果 $a[w] == b[h]$, 那么从 $(w-1, h-1)$ 到 (w, h) 的时候所有 i 都 $+1$ 了
- 那么 $\text{pre}[w][h]$ 就是 $\text{suf}[w][h-1]$ 的补, $\text{suf}[w][h]$ 就是 $\text{pre}[w-1][h]$ 的补

- 同样 $l++$ 的时候, 取 rightmost 的路径可以尽量从 $(j, l-1)$ 转移过来, 所以 $l++$ 的时候对应的是一个前缀
- 同样 $l++$ 的时候, 取 rightmost 的路径可以尽量从 $(j, l-1)$ 转移过来, 所以 $l++$ 的时候对应的是一个前缀
- 我们把 $(w, h-1) \rightarrow (w, h)$ 的 $+1$ 前缀称为 $\text{pre}[w][h]$, 同样的 $(w-1, h) \rightarrow (w, h)$ 的 $+1$ 后缀称为 $\text{suf}[w][h]$
- 然后如果 $a[w] == b[h]$, 那么从 $(w-1, h-1)$ 到 (w, h) 的时候所有 i 都 $+1$ 了
- 那么 $\text{pre}[w][h]$ 就是 $\text{suf}[w][h-1]$ 的补, $\text{suf}[w][h]$ 就是 $\text{pre}[w-1][h]$ 的补
- 如果 $a[w] \neq b[h]$, 那么显然 $f[l][i][j] = \max(f[l-1][i][j], f[l][i][j-1])$

- 那么我们画画就可以明白，如果 $\text{pre}[w - 1][h]$ 和 $\text{suf}[w][h - 1]$ 的并是全集，显然 $\text{suf}[w][h] = \text{pre}[w - 1][h]$ 的补，否则等于 $\text{suf}[w][h - 1]$ ，把它写成 $\max(\text{pre}[w - 1][h] + 1, \text{suf}[w][h - 1])$ 就好了

- 那么我们画画就可以明白，如果 $\text{pre}[w-1][h]$ 和 $\text{suf}[w][h-1]$ 的并是全集，显然 $\text{suf}[w][h] = \text{pre}[w-1][h]$ 的补，否则等于 $\text{suf}[w][h-1]$ ，把它写成 $\max(\text{pre}[w-1][h] + 1, \text{suf}[w][h-1])$ 就好了
- $\text{pre}[w][h]$ 类似

- 那么我们画画就可以明白，如果 $\text{pre}[w-1][h]$ 和 $\text{suf}[w][h-1]$ 的并是全集，显然 $\text{suf}[w][h] = \text{pre}[w-1][h]$ 的补，否则等于 $\text{suf}[w][h-1]$ ，把它写成 $\max(\text{pre}[w-1][h] + 1, \text{suf}[w][h-1])$ 就好了
- $\text{pre}[w][h]$ 类似
- 推出来所有 suf 和 pre 之后，直接在最后一行用 suf 算出每个 $f[i][j]$ 即可

- 那么我们画画就可以明白，如果 $\text{pre}[w-1][h]$ 和 $\text{suf}[w][h-1]$ 的并是全集，显然 $\text{suf}[w][h] = \text{pre}[w-1][h]$ 的补，否则等于 $\text{suf}[w][h-1]$ ，把它写成 $\max(\text{pre}[w-1][h] + 1, \text{suf}[w][h-1])$ 就好了
- $\text{pre}[w][h]$ 类似
- 推出来所有 suf 和 pre 之后，直接在最后一行用 suf 算出每个 $f[i][j]$ 即可
- 主要就是利用 $+1$ 的单调性，把 i 的一维压到值里，减去一维